# Technical Documentation

## 1. Introduction

### What is RAG?

Retrieval-Augmented Generation (RAG) is an AI paradigm that combines **retrieval from a knowledge base** with **generation by a language model**. Instead of relying solely on the LLM's internal parameters, RAG grounds responses in factual documents. This reduces hallucinations and ensures accuracy.

### Why RAG is Needed?

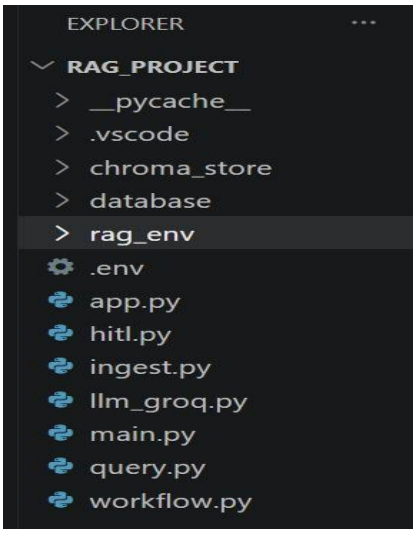
Customer support requires **trustworthy, contextual answers**. Traditional chatbots often fail when asked about specific company policies or technical details. By integrating RAG, the assistant retrieves relevant chunks from a PDF knowledge base before generating answers.

### Use Case Overview

This project implements a **Customer Support Assistant** that:

- Ingests a PDF knowledge base.
- Stores embeddings in ChromaDB.
- Retrieves relevant chunks for queries.
- Uses Groq LLM for contextual answers.
- Employs LangGraph for workflow orchestration.
- Escalates unresolved queries to human agents via HITL.

## 2. System Architecture Explanation



**Layers**

1. **User Interface:** CLI and Streamlit UI.
2. **Document Processing:** PDF Loader + Chunker.
3. **Embedding Generation:** HuggingFace MiniLM-L6-v2.
4. **Vector Storage:** ChromaDB.
5. **Retrieval Layer:** Similarity search.
6. **LLM Processing:** Groq LLaMA 3.
7. **Workflow Orchestration:** LangGraph nodes.
8. **Routing Layer:** AI vs HITL.
9. **HITL System:** Human escalation.

**Component Interactions**

- User submits query → converted to embedding.
- Retriever fetches chunks → passed to Groq LLM.
- LangGraph decides whether AI or HITL answers.
- Final response returned to user.

## 3. Design Decisions

**Chunk Size Choice**

- **1000 characters, 200 overlap** chosen to balance retrieval accuracy and context.
- Larger chunks risk irrelevant context; smaller chunks risk fragmented answers.

**Embedding Strategy**

- HuggingFace MiniLM-L6-v2 used for embeddings.
- Free, local, lightweight, avoids API dependency.

**Retrieval Approach**

- ChromaDB chosen for persistence and semantic search.
- Top-k similarity search ensures relevant chunks.

**Prompt Design Logic**

- System prompt: "You are a helpful customer support assistant."
- User prompt includes retrieved context + query.
- Ensures answers are grounded in factual data.

## 4. Workflow Explanation

**Node Responsibilities**

- **Input Node:** Accepts query.
- **Router Node:** Determines intent and confidence.
- **Answer Node:** Calls Groq LLM.
- **HITL Node:** Escalates to human agent.
- **Output Node:** Formats final response.

**State Transitions**

- Confidence $\geq 0.7 \rightarrow$ Answer Node.
- Confidence $< 0.7 \rightarrow$ HITL Node.

## 5. Conditional Logic

**Intent Detection**

- Queries classified as **answerable** or **escalate**.
- Confidence score determines routing.

**Routing Decisions**

- **Answerable Queries:** Routed to Groq LLM.
- **Low Confidence Queries:** Routed to HITL.
- **Complex Queries:** Routed to HITL.
- **Missing Context:** Routed to HITL.

## 6. HITL Implementation

**Role of Human Intervention**

- Ensures accuracy when AI confidence is low.
- Provides answers for queries outside knowledge base.
- Updates knowledge base for future queries.

**Benefits**

- Reliability: Human ensures correctness.

- Adaptability: Handles queries beyond AI scope.
- Continuous Improvement: Human answers can be added to knowledge base.

**Limitations**

- Slower response time.
- Requires human availability.

## 7. Challenges & Trade-offs

**Retrieval Accuracy vs Speed**

- Larger chunk size improves context but slows retrieval.
- Smaller chunk size speeds retrieval but risks fragmented answers.

**Chunk Size vs Context Quality**

- 1000/200 chosen as compromise.

**Cost vs Performance**

- HuggingFace embeddings are free but slower than proprietary APIs.
- Groq LLM provides fast inference but requires API integration.

**Model Deprecation**

- Groq model llama3-8b-8192 was decommissioned.
- Updated to llama3-70b-8192.

## 8. Testing Strategy

**Approach**

- Test queries against knowledge base.
- Compare AI answers with handbook.
- Escalate low-confidence queries to HITL.

**Sample Queries**

1. **SLA for Priority 1 incidents** → "Target response time is 1 hour."
2. **Escalation process** → "Escalated to 2nd/3rd line support."
3. **Support hours for US customers** → "Monday–Friday, 8:30 AM–5:30 PM EST."
4. **Maintenance renewals** → "Customers lose support if not renewed."

## 9. Future Enhancements

**Multi-Document Support**

Enable ingestion of multiple PDFs.

**Feedback Loop**

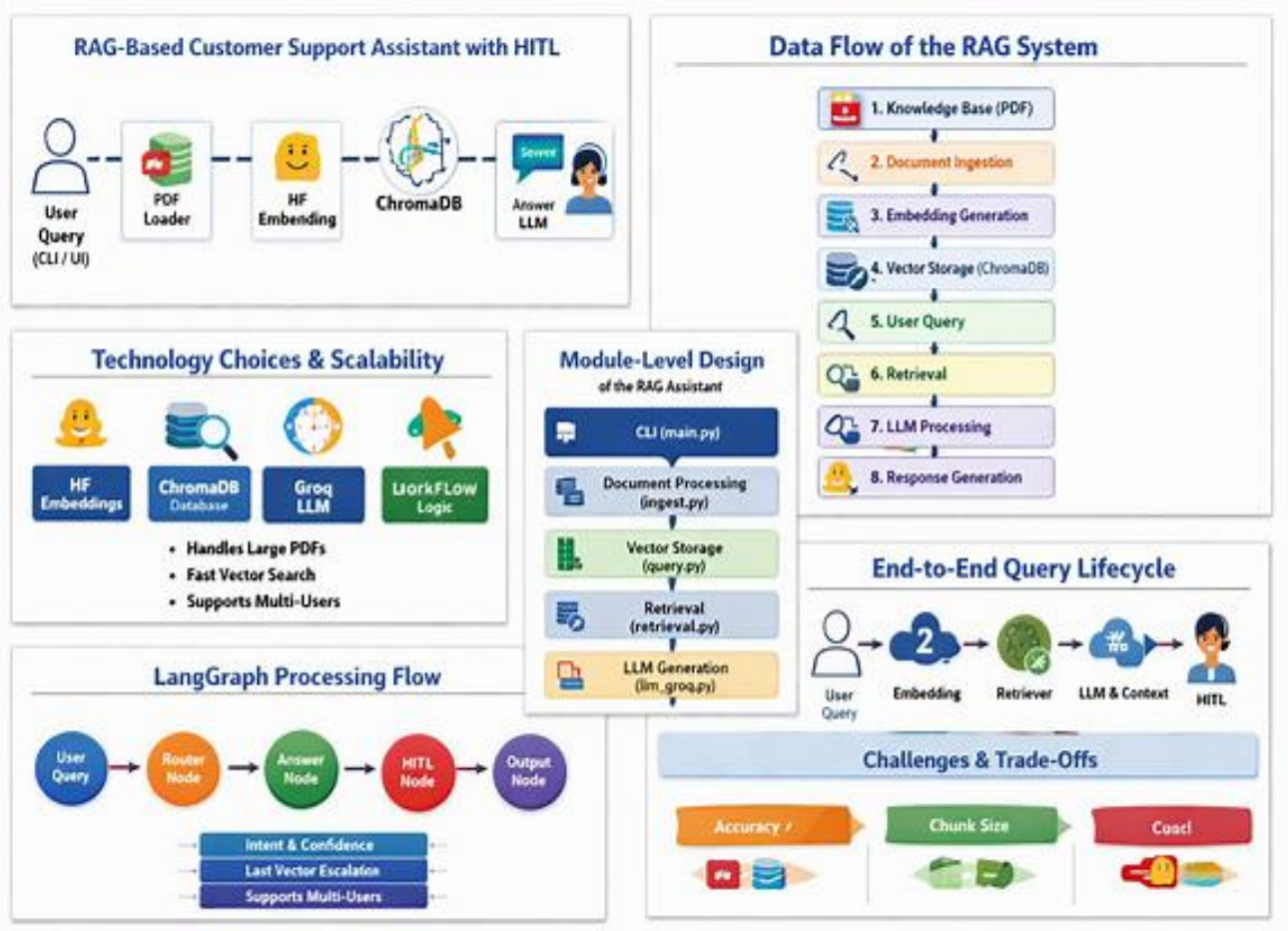
Integrate user feedback to improve retrieval accuracy.

**Memory Integration**

Store past queries and answers for personalization.

**Deployment**

Deploy to cloud for scalability and multi-user support.




# Conclusion

The Technical Documentation provides a comprehensive explanation of the RAG-based Customer Support Assistant. By detailing system architecture, design decisions, workflow, conditional logic, HITL implementation, challenges, testing, and future enhancements, this document ensures that engineers can understand, replicate, and extend the system.The assistant demonstrates how RAG, LangGraph, and HITL can be combined to build a scalable, reliable, and context-aware customer support solution.